Тема 11: Процедурно програмиране – основни конструкции.

ТЕОРИЯ

1. Принципи на структурното програмиране

Процедурно програмиране: изпълняваме инструкции стъпка по стъпка; програмата се разделя на **процедури/функции**.

Структурното програмиране е частен случай:

- Без `goto` — управлението на потока само чрез **последователност, разклонение, цикъл**
- Един вход и един изход за всеки блок
- Йерархична структура на програмата

3-те основни управляващи конструкции:

1. **Последователност** — изпълнение ред по ред
2. **Разклонение (selection)** — `if/else`, `switch`
3. **Итерация (iteration)** — `for`, `while`, `do/while`

---

2. Управляващи конструкции в C++

Условни оператори

```
// if / else if / else
if (условие1) {
    // ако условие1 е true
} else if (условие2) {
    // ако условие1 е false, условие2 е true
} else {
    // ако всички условия са false
}

// switch (само целочислени типове и char)
switch (израз) {
    case K1: операция; break;
    case K2: операция; break;
    default: операция; break;   // незадължителен
}
// ВАЖНО: без break → fall-through (изпълняват се и следващите case-ове)

// Тернарен оператор
int max = (a > b) ? a : b;
```

Оператори за цикъл

```
// while – проверява условието ПРЕДИ тялото (0 или повече изпълнения)
while (условие) {
    // тяло
}

// do/while – проверява условието СЛЕД тялото (поне 1 изпълнение)
do {
    // тяло
} while (условие);

// for – init; условие; корекция
for (int i = 0; i < n; i++) {
    // тяло
}
// i е видима само в scope-а на for!

// break  → излиза от цикъла/switch
// continue → преминава към следващата итерация
```

---

3. Променливи

Синтаксис: `<тип> <име> [= <начална стойност>]`

```
int a;          // НЕИНИЦИАЛИЗИРАНА – стойността е непредвидима!
int b = 10;
double d = 3.14;
const int N = 5; // константа – не може да се променя
```

Вид	**Дефиниране**	**Видимост**	**Жизнен цикъл**
Локална	В тялото на функция/блок	Само в своя scope	От дефинирането до края на блока
Глобална	Извън всяка функция	От всички функции	До края на програмата

```
int global = 10;   // глобална

void f() {
    int local = 5;          // локална
    cout << global << endl; // 10 – достъпна
    // local не е достъпна извън f()
}
```

Оператор за присвояване =:

- Лявата страна: **lvalue** (именувана памет — промен., елемент на масив)
- Дясната страна: **lvalue или rvalue** (константа, литерал, резултат)
- Връща референция към лявата страна → **дясно-асоциативен**: `a = b = c = 4`

---

4. Типове в C++

Тип	Размер	Стойности
`bool`	1 байт	true/false
`char`	1 байт	-128..127 или символи
`int`	4 байта	~±2 млрд
`long long`	8 байта	~$\pm 9 \times 10^{18}$
`float`	4 байта	~7 знак. цифри
`double`	8 байта	~15 знак. цифри
`void`	—	Няма стойност

Имплицитно преобразуване: при несъответствие на типове компилаторът преобразува автоматично (може да загуби точност):

```
int i = 3.7;    // i = 3 (truncation!)
double d = 5;   // d = 5.0 (OK – разширяване)
```

---

5. Функции

```
<тип_на_връщане> <име>(<параметри>) {
    // тяло
    return стойност; // ако не е void
}
```

Декларация (прототип) — преди употреба:

```
int max(int a, int b);  // декларация
// ...
int max(int a, int b) { return a > b ? a : b; }  // дефиниция
```

return: прекратява изпълнението на функцията и връща резултата. В `void` функция: само `return;`

---

6. Видове предаване на параметри

По стойност (копие)

```
void f(int a) { a = 100; }  // промяната НЕ се отразява на оригинала
int x = 5;
f(x);  // x остава 5
f(5);  // OK – приема rvalue
```

По референция

```
void f(int& a) { a = 100; }  // промяната СЕ отразява на оригинала
int x = 5;
f(x);   // x става 100 ✓
// f(5); – ГРЕШКА! Очаква lvalue
```

По константна референция (ефективно четене без копие)

```
void f(const int& a) { /* не може да променя a */ }
f(x);   // OK
f(5);   // OK — може rvalue
```

По указател

```
void f(int* ptr) { *ptr = 100; }
int x = 5;
f(&x);   // x става 100 ✓
```

Сравнение:

Метод	Синтаксис	Промяна оригинала	Приема rvalue
По стойност	`int a`	✗	✓
По референция	`int& a`	✓	✗
По const ref	`const int& a`	✗	✓
По указател	`int* p`	✓ (чрез `*p`)	✗

---

7. Символни низове в C

```
char str[10] = "abc";
// Памет: ['a']['b']['c']['\0']['\0']['\0']['\0']['\0']['\0']['\0']
```

Задължително завършват с `\0` (терминираща нула)!

Основни функции (`<cstring>`):

```
strlen(str)              // дължина без \0
strcpy(dest, src)        // копира src в dest (включва \0)
strcat(dest, src)        // конкатенира src накрая на dest
strcmp(a, b)             // 0 ако равни, <0 ако a<b, >0 ако a>b
strncpy(dest, src, n)    // копира най-много n символа
```

Динамичен низ:

```
char* str = new char[strlen(src) + 1];  // +1 за \0
strcpy(str, src);
// ... употреба ...
delete[] str;
```

КАК СЕ РЕШАВАТ ЗАДАЧИ

Тема 11 е директно свързана с **Task 1** на практическия изпит (процедурен C++).
Формати: **попълни празните места** в код, **проследи изхода**, напиши функция.

Тип 1: Проследяване на изход (Trace)

Стратегия: изпълнявай мислено ред по ред, следи стойностите на променливите.

```
int x = 5, y = 3;
if (x > y) {
    cout << x - y;   // 2
} else {
    cout << y - x;
}
// Изход: 2
```

При цикли — таблица на стойностите:

```
int s = 0;
for (int i = 1; i <= 4; i++) {
    s += i;
    cout << s << " ";
}
// i:  1   2   3   4
// s:  1   3   6   10
// Изход: 1 3 6 10
```

Тип 2: Попълване на празни места

Стратегия: разбери какво прави функцията от контекста и примерите → намери липсващата логика.

```
// Пример: функция, която преброява четните елементи
int countEven(int* arr, int n) {
    int count = 0;
    for (int i = 0; i < ___; i++) {   // n
        if (arr[i] ___ 2 == 0) {       // %
            ___;                        // count++
        }
    }
    return count;
}
```

Чести попълвания:

- Условие за спиране на цикъл: `i < n`, `i <= n-1`, `*ptr != '\0'`
- Оператор: `%`, `==`, `!=`, `++`, `--`
- Return стойност: `result`, `true/false`, `max/min`

Тип 3: Написване на функция

Шаблон за мислене:

1. Каква е сигнатурата? (тип на връщане, параметри)
2. Трябва ли да модифицира оригинала? → референция/указател
3. Итерация: `for`, `while`, или рекурсия?

4. Какво се връща?

Честа задача — намери максимален елемент в масив:

```
int findMax(int* arr, int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) max = arr[i];
    }
    return max;
}
```

Честа задача — обърни низ:

```
void reverseStr(char* str) {
    int len = strlen(str);
    for (int i = 0; i < len / 2; i++) {
        char tmp = str[i];
        str[i] = str[len - 1 - i];
        str[len - 1 - i] = tmp;
    }
}
```

Честа задача — провери дали е палиндром:

```
bool isPalindrome(const char* str) {
    int l = 0, r = strlen(str) - 1;
    while (l < r) {
        if (str[l] != str[r]) return false;
        l++; r--;
    }
    return true;
}
```

Бърза справка

Конструкция	**Ключов факт**
`while`	Проверява ПРЕДИ — може 0 пъти
`do/while`	Проверява СЛЕД — поне 1 път
`for`	`init; condition; step` — i видима само в for
`switch`	Без `break` → fall-through!
По стойност	Копие — оригинала непроменен
По референция `&`	Директен достъп — оригинала се мени
Неинициализирана локална	Стойността е мусор!
Глобална	Видима навсякъде, жива до края
`strlen`	Без `\0`; масивът трябва да завършва с `\0`
`strcpy(dst, src)`	`dst` трябва да е достатъчно голям (+1 за `\0`)